

NumPy 数组与数值计算 实验手册

第五讲 配对不变的数组计算

本实验把第三讲的四套房预测改写为 NumPy 数组运算，并检验向量化是否保持样本对应关系。你会亲手重现一个能正常运行却给出错误分数的广播问题，再为函数加入形状检查。实验只用 CPU 和本地合成数据，不访问网络、不调用 API，也不需要显卡。

完成后应留下三样东西：一份从头顺序执行并保存输出的 Notebook、一份通过检查的脚本结果、一段对错误广播的解释。解释必须包括输入形状、错误输出形状、正确配对与错误配对的区别。运行时间是观察结果，不要求达到固定加速倍数。

一 文件与运行环境

本目录中的 `lecture.md` 是主讲稿，`answers.md` 是练习详解；`experiment.ipynb` 是逐步实验，`experiment.py` 是可以独立执行的完整检查脚本。`data` 目录保存对应的三个小型 CSV 和数据字典，`figures` 是讲稿插图。代码直接写明教学数值，避免本讲提前依赖 CSV 读取；你可以打开 CSV 逐行核对样本顺序。

先启动已在第三、四讲使用的 Anaconda 环境，再从 Anaconda 的终端进入本讲文件夹。不同系统终端外观可能不同，但 Python 命令含义相同。下面的命令在终端执行，不是在 Notebook 的 Python 代码格里执行：

```
python -c "import sys, numpy; print(sys.version); print(numpy.__version__)"
```

应看到 Python 和 NumPy 的版本号。课程核验使用 Python 3.12.14、NumPy 2.3.5；如果本机版本不同，先记录版本，不要急着更换整个环境。关键行为以本讲的测试为准。若提示找不到 `numpy`，确认当前终端和 Notebook 选择的是同一个科学计算环境，再按你已安装的 Anaconda 环境管理方式安装 NumPy。不要在不明环境里连续试装多个版本。

正常学习与运行实验只需要 NumPy。查看 Notebook 需要本机已有 JupyterLab 或 Notebook。重新生成讲稿插图才需要 `Matplotlib` 和中文字体，不是学习者完成数值实验的必要步骤。本次已验证脚本、数值输出与真实 Python 内核的顺序执行；没有在你的电脑上测试 Anaconda 安装、浏览器 Jupyter 界面或内核选择菜单。

打开 `experiment.ipynb` 后，先保存一个自己的练习副本。使用“重启内核并运行全部”或界面中含义相同的操作，从第一格按顺序运行。第一次可以逐格执行并停下来手算，最终验收仍要重启后全量执行。若跳过某格也能运行，可能只是内核留着旧变量，不代表文件独立完整。

二 在运行前先写下答案

把下面两组答案写在纸上，暂时不要看输出。

第一组是旧房价任务。面积 [50, 60, 80, 90] 平方米，标签 [110, 130, 170, 190] 万元。A 是 $2 \times x$ ，B 是 $2 \times x + 10$ ，C 是 $1.5 \times x + 40$ 。写出 C 的四个预测、四个带符号误差、四个绝对误差及平均数。预期每个逐样本数组形状均为 (4,)。

第二组是新合成数据。三行特征为 `[[50,1],[60,2],[80,1]]`，第一列平方米，第二列房间数。面积系数为 2 万元每平方米，房间系数为 5 万元每间，公共偏置 10 万元。写出三行的特征贡献和最终预测。这个规则是提前指定的教学规则，不是从数据学出的规律。

在任何代码输出之前写下答案，有助于防止“程序显示什么就接受什么”。如果你只在运行之后把结果抄回纸上，就失去独立参考的作用。纸笔结果也可能错，但独立路线出现差异时，至少能告诉我们有一处需要解释。

三 从列表变成数组

在 Notebook 的数组单元中运行：

```
import numpy as np
area = np.array([50.0, 60.0, 80.0, 90.0])
target = np.array([110.0, 130.0, 170.0, 190.0])
print(area.shape, area.ndim, area.size, area.dtype)
```

应得到形状 (4,)、轴数 1、元素数 4、数据类型 float64。如果数组被写成 `[[50,60,80,90]]`，形状会是 (1,4)；如果写成 `[[50],[60],[80],[90]]`，形状会是 (4,1)。数字没有变，结构已经不同。本实验评分接口要求单输出预测与标签保持 (n,)。

再比较 `[50,60] * 2` 和 `np.array([50,60]) * 2`。前者重复列表，后者逐元素乘 2。请用一句话解释为什么把一段旧代码的列表替换成数组之后，仍然需要重新核对运算语义。

如果输出是字符串类型，检查输入中是否混入了带引号的数字或“未知”等文字。不要为了让运算继续而随意丢掉这些位置。本实验数据没有缺失，出现此类情况应先修正输入；真实缺失数据将在下一讲处理。

四 循环与向量化逐项比较

先保留一个循环预测：

```
loop_prediction = []
for value in area:
    loop_prediction.append(1.5 * float(value) + 40.0)
loop_prediction = np.array(loop_prediction)
array_prediction = 1.5 * area + 40.0
assert loop_prediction.shape == array_prediction.shape == (4,)
np.testing.assert_allclose(array_prediction, loop_prediction,
                           rtol=0.0, atol=1e-12)
```

`float(value)` 把取出的数组数值转换为普通 Python 浮点值，使循环参考的标量运算路线清楚。
`assert_allclose` 不打印东西通常意味着检查通过，不是代码没有执行。若不满足容差，它会产生详细失败信息。注意我们在比较数值前先比较了形状。

依次计算预测减标签、绝对值和平均数。C 的预期结果为 `[115,130,160,175]`、`[5,0,-10,-15]`、`[5,0,10,15]`、7.5。再运行三规则比较，确认训练 MAE 的字典显示 A 为 10、B 为 0、C 为 7.5。

然后核对既定测试。面积 65、85 上，B 的预测是 140、180；标签为 144、174；绝对误差为 4、6，平均为 5。训练标签均值为 150，固定基线测试 MAE 为 15。这里只复核先前已确定结果，不用测试答案改变规则，也不把本讲披露过的数据继续当成未知封存集。

暂停思考：假如循环预测的顺序被颠倒，而数组预测保持原顺序，只比较预测均值能发现吗？通常不能，因为重新排列不改变均值。比较逐项结果才与样本对应关系有关。

五 让形状预测先于运行

构造二维 `x`，先在纸上填出下面的形状，再运行确认：`x` 是 `(3,2)`，`x[0]` 是 `(2,)`，`x[:1]` 是 `(1,2)`，`x[:,0]` 是 `(3,)`，`x[:,1]` 是 `(3,1)`。

```
X = np.array([[50., 1.], [60., 2.], [80., 1.]])
for name, value in [("X", X), ("X[0]", X[0]),
                    ("X[:1]", X[:1]), ("X[:,0]", X[:,0]),
                    ("X[:,1]", X[:,1])]:
    print(name, value.shape)
```

如果你写出 `x[0]` 的形状是 `(1,2)`，回看讲稿关于整数索引与范围切片的区别。预测函数要处理一批样本，因此输入一个样本时推荐 `x[:1]`。不要为通过某一次运行而把函数改成同时猜测多种不明确含义。

归约练习使用无单位的 `Z = [[1,2],[3,4],[5,6]]`。`sum(axis=0)` 应得到 `[9,12]`，`sum(axis=1)` 得到 `[3,7,11]`。再打开 `keepdims=True`，查看形状分别变为 `(1,2)` 与 `(3,1)`。请在记录中写“哪个轴被合并”，不要只写“按行”或“按列”。

最后用 `X * np.array([2.,5.])` 查看三行贡献，沿 `axis=1` 求和后加 10。输出应是 `[115,140,175]`，形状 `(3,)`。它与主讲稿的循环函数、纸笔答案都应一致。面积和房间数不能直接相加，贡献能相加是因为系数已经把两者变成同样的万元单位。

六 重现一个不会自动报错的错误

这一步使用新的 `p` 与 `y`，不会修改前面的数据。先算正确 MAE，再故意把预测增加一条轴：

```
p = np.array([115., 140., 175.])
y = p.copy()
correct = np.abs(p - y).mean()
wrong_difference = p[:, None] - y
wrong = np.abs(wrong_difference).mean()
print(correct, wrong_difference.shape, wrong)
print(wrong_difference)
```

正确结果为 0。错误差值形状为 (3,3)，内容是三行 [0,-25,-60]、[25,0,-35]、[60,35,0]，错误均值约 26.6667。请指出第 0 行第 2 列的 -60 来自什么：它是第一套房的预测 115 减第三套房的标签 175。它不是第一套房自己的误差。

然后使用 mae_checked:

```
from experiment import mae_checked
try:
    mae_checked(p[:, None], y)
except ValueError as error:
    print("已按接口要求拒绝: ", error)
else:
    raise AssertionError("错误形状没有被拒绝")
```

看到“已按接口要求拒绝”是这一格的正确结果。try 只捕获这一步预期出现的 ValueError；如果没有报错，else 会明确让测试失败。不要用一个包围整本 Notebook 的宽泛异常捕获来掩盖错误，也不要删掉检查让它“顺利结束”。

再分别传入一个二维列状标签、长度不同的标签和两个空数组，确认函数拒绝。把错误输入修回一维同形状后，应恢复正常 MAE。这里的修复依据是接口约定，不是随意试几个 reshape。

七 非方阵 单样本 与不修改输入

运行带形状契约的双特征函数，先保存一份输入副本：

```
from experiment import weighted_predict
before = X.copy()
pred = weighted_predict(X, [2., 5.], 10.)
one = weighted_predict(X[:,1], [2., 5.], 10.)
assert pred.shape == (3,)
assert one.shape == (1,)
np.testing.assert_allclose(pred, [115., 140., 175.], rtol=0, atol=1e-12)
np.testing.assert_array_equal(X, before)
```

然后只保留面积特征 `X[:,1]`，系数改成 [2.]，预测应是 [110,130,170]。保留第二个特征轴是必要的，否则一维输入不符合双特征函数的一般契约。

故意传入 `x[0]`、长度错误的系数、`np.empty((0,2))`、包含 `np.nan` 的输入，以及列表形式的偏置 `[10.]`。每一种都应得到清楚的 `ValueError`。最后一种检查是为了要求公共偏置为单个数，而不是一个可能继续广播的数组。

为什么要用三行两列？如果只用两行两列，某些轴交换错误依然保留相同形状，可能侥幸通过形状检查。为什么还要单样本？因为整数索引可能移除样本轴。不同测试互相补充，任何单一形状都不能覆盖所有风险。

八 视图和数值类型的小实验

为视图实验新建数组，不要直接修改 `x`。分别创建基本切片、显式副本和高级索引结果，使用 `np.shares_memory` 检查。修改切片后，原数组应随之变化，其他副本不变。请在记录里写出“哪个索引对应同一处存储”，而不只是写“NumPy 有坑”。

整数实验采用两个明确 `int8` 的数组。`[120]+[10]` 得到 -126，而先转换为 `int64` 再算得到 130。这是故意触发的小范围演示，不可把它当作正常金额算法。再把 1.9 写到整数数组某个位置，确认保存的是 1，并解释为什么需要在赋值前选择浮点类型。

浮点实验先创建 `v = np.array([100000000.,1.,-1000000000.], dtype=np.float32)`。默认求和在本讲环境中得到 0，指定 `dtype=np.float64` 累加得到 1。再把已经按 `float32` 保存的 1000000001 转为 `float64`，观察精度损失并没有恢复。一个实验关注中间累加，另一个关注输入保存，两者不要混淆。

九 测量性能并写出边界

Notebook 调用与脚本相同的 `timing_experiment()`。它比较 4、100、10000、100000 个样本；每种规模有预热和五组重复测量，报告每次调用毫秒数的中位数。两边都执行预测、绝对误差和平均值，输入准备放在计时外。

不要把任何一组耗时写成固定标准答案，也不要因为小数组上没有加速而判实验失败。你要记录：样本数、循环时间、数组时间、版本、输入准备是否计入、重复次数。若某次波动明显，重新运行并比较，但不能只挑最好看的一次冒充稳定结果。

独立脚本在终端运行：

```
python experiment.py --output my-test-result.json
```

应显示 `status` 为 `passed`、`checks` 为 10，并生成你命名的 JSON 结果。若只想检查数值与边界、不做计时，可以执行：

```
python experiment.py --skip-timing --output my-checks-only.json
```

命令中的 JSON 文件保存在当前终端目录；如果你从本讲文件夹运行，就会与脚本同目录。使用不同文件名可保留不同运行记录。不要把尚未运行的旧输出文件作为本次验收证据。

十 常见故障与处理顺序

遇到 `NameError`，先检查是否跳过了前面的定义格，重启并从头运行。遇到 `ModuleNotFoundError`，检查 Notebook 与终端环境，以及是否在本讲目录中导入 `experiment.py`。不要把相同名字的个人脚本命名成 `numpy.py`，那可能遮住真正的 NumPy 库。

遇到广播 `ValueError`，打印操作数的形状，按最右轴开始核对，不要先试转置。遇到 `AxisError` 或提示轴超出范围，通常意味着你对一维数组使用了 `axis=1`，应追查哪次索引去掉了轴。遇到输出形状变成 `(n,n)`，先看是否混用了列状预测和一维标签。

遇到 `AssertionError`，看失败的是形状、数值还是输入未修改。先恢复原始小数据，与纸笔答案比较，再考虑浮点容差。不要为了让测试通过而扩大到没有意义的容差。若输出含 `nan`，检查空数组、无效数据和非法运算；它不是一个“比较大的预测误差”。

十一 提交检查清单

- Notebook 以新内核从上到下执行，保留每格输出，没有未处理异常。
- 脚本在新进程中执行，10 组检查通过，记录 NumPy 与 Python 版本。
- 三规则训练 MAE 为 10、0、7.5；既定测试 B 为 5，常数基线为 15。
- 非方阵预测为 `[115,140,175]`；单样本输出 `(1,)`；单特征输入仍保留二维结构。
- 能解释错误广播的九个差值，并确认检查函数拒绝错误形状。
- 能说明视图修改为何影响原数组，以及提高存储精度为何不能找回已丢掉的信息。
- 性能结论写出计时边界与环境，没有宣称 NumPy 在所有任务上固定快多少倍。

最后用三句话总结：本次最容易忽略的形状是什么；你用哪个测试发现它；这个测试仍然没有证明什么。带着这些边界意识进入下一讲，比背下一页 API 名称更重要。